Will write code that writes code that writes code for food.

➤ *Martin Rodgers*

Introduction

Metaprogramming…it sounds cool! It sounds like a design technique for high-level enterprise architects or a faddish buzzword that has found its way into press releases.

In fact, far from being an abstract concept or a bit of marketing-speak, metaprogramming is a collection of down-to-earth, pragmatic coding techniques. It doesn't just sound cool; it *is* cool. Here are some things you can do with metaprogramming in the Ruby language:

- Say you want to write a Ruby program that connects to an external system—maybe a web service or a Java program. With metaprogramming, you can write a wrapper that takes *any* method call and routes it to the external system. If somebody adds methods to the external system later, you don't have to change your Ruby wrapper; the wrapper will support the new methods right away. That's magic.
- Maybe you have a problem that would best be solved with a programming language that's specific to that problem. You could go to the trouble of writing your own language, custom parser and all. Or you could just use Ruby, bending its syntax until it looks like a specific language for your problem. You can even write your own little interpreter that reads code written in your Ruby-based language from a file.
- You can aggressively remove duplication from your Ruby code while keeping it elegant and clean. Imagine twenty methods in a class that all look the same. How about defining all those methods at once, with just a few lines of code? Or maybe you want to call a sequence of similarly named methods. How would you like a single short line of code that calls all the methods whose names match a pattern—like, say, all methods that begin with *test*?
- You can stretch and twist Ruby to meet your needs, rather than adapt to the language as it is. For example, you can enhance any class (even a core class like `Array`) with that method you miss so dearly, you can wrap